Введение в Python

В рамках данного проекта ты освежишь знания основ языка Python и применишь их на практике. Тебе предстоит разработать простую систему поиска текста, а также освоить базовые методы анализа текстовых данных. В процессе работы ты также познакомишься с интерактивной средой Jupyter и настройкой необходимого окружения Python.

Оглавление

1. Глава 1
   1.1 Преамбула
2. Глава 2
   2.1 Цели
3. Глава 3
   3.1 Задание
4. Глава 4
   4.1 Сдача работы и проверка

Глава 1

Преамбула

За последние годы язык программирования Python стал одним из ключевых инструментов для анализа и визуализации больших массивов данных. Благодаря своей читабельности, простому синтаксису и отсутствию необходимости в компиляции, Python широко используется как в учебных целях, так и в профессиональной среде. Он позволяет сосредоточиться на изучении алгоритмов, концепций и парадигм программирования, не перегружая разработчика техническими деталями.

Популярность Python в сфере Data Science объясняется в первую очередь богатой экосистемой специализированных библиотек и активным сообществом разработчиков. Среди наиболее значимых инструментов можно выделить:

- **NumPy** — для работы с многомерными массивами и векторизованными вычислениями;
- **Pandas** — для обработки и анализа табличных и разнородных данных;
- **SciPy** — для решения задач научных и инженерных вычислений;
- **Matplotlib** — для построения графиков и профессиональной визуализации данных;
- **Scikit-Learn** — для решения задач классического машинного обучения;
- **OpenCV** — для компьютерного зрения: обработки изображений, распознавания объектов и видеоаналитики;
- **PyTorch** — для разработки и обучения нейронных сетей.

Однако применение Python не ограничивается только анализом данных. Он активно используется:

- Для **разработки сервисов** и **приложений** (например, создания REST API с помощью FastAPI или Django);

- Для **автоматизации тестирования** и обеспечения качества кода (библиотеки pytest, unittest и инструменты для интеграционного тестирования).

> Этот проект — твоя отправная точка. Он поможет тебе познакомиться или закрепить основы синтаксиса и работы с ключевыми библиотеками Python, научит работать с данными и писать первые алгоритмы. Все последующее обучение — от анализа данных до внедрения моделей — будет опираться на эти навыки. Поэтому важно пройти этот этап внимательно: он формирует фундамент, на котором строится весь курс.

Глава 2

Цели

В этом проекте мы предлагаем тебе освежить знания по Python и научиться применять их для решения реальных задач. Ты разберешься с базовыми структурами данных, циклами, условиями, функциями и классами — всем тем, что составляет фундамент программирования.

Чтобы закрепить навыки, ты реализуешь небольшой алгоритм. В качестве примера мы возьмем инвертированный индекс — задачу обработки текстовых данных и поиска информации, которая поможет вспомнить базовый синтаксис Python.

Задания этого проекта позволят не только закрепить базовые навыки работы с Python, но и понять, что разнообразные прикладные задачи можно эффективно решать без сложных моделей машинного обучения и искусственного интеллекта.

Глава 3

Задание

Задание 0. Подготовка окружения

Чтобы эффективно работать с Python, особенно в проектах по Data Science, важно уметь создавать и управлять виртуальными окружениями. Виртуальное окружение позволяет:

- Изолировать зависимости проекта — разные проекты могут использовать разные версии библиотек, не мешая друг другу;
- Избегать конфликтов версий Python и пакетов;
- Легко переносить и разворачивать проекты на других машинах.

В Python есть несколько способов создать виртуальное окружение, но самым популярным является использование встроенного модуля venv.

Задание 0.1. Создание виртуального окружения

- Создай виртуальное окружение с именем project1.
- Активируй виртуальное окружение.
- Выведи список библиотек, установленных в новом окружении.
- Сделай скриншот командной строки, чтобы было видно и активное окружение, и список библиотек. Сохрани его в папке assets.

Задание 0.2. Установка библиотек

Для установки библиотек используется пакетный менеджер pip, который позволяет скачивать, обновлять и управлять зависимостями проекта внутри виртуального окружения.

- Убедись, что твое виртуальное окружение (project1) активировано.
- Установи библиотеку jupyterlab.
- Сохрани список всех установленных библиотек и их версий в файл requirements.txt.

  Важно: загружать само виртуальное окружение на Git не требуется — достаточно хранить файл requirements.txt в корне проекта, чтобы его можно было использовать для повторного создания окружения.

Jupyter Notebooks

> **Jupyter Lab** — это мощная среда для интерактивной работы с Python и другими языками программирования, которая особенно полезна для анализа данных. Основные преимущества Jupyter Lab:

- Позволяет запускать код по ячейкам, что удобно для пошаговой проверки результатов;
- Можно сочетать код, текст, формулы (Markdown и LaTeX) и графики в одном документе;
- Широко используется в Data Science и машинном обучении для экспериментов и визуализации данных;
- Поддерживает расширения для работы с Git, визуализацией и интерактивными виджетами.

Запуск Jupyter Lab в активированном виртуальном окружении:

```
jupyter lab
```

После запуска откроется веб-интерфейс, где можно создавать и редактировать jupyter-ноутбуки.

Всю работу над проектом ты будешь вести в ноутбуке 01-assignment.ipynb, который находится в папке src. Это твоя основная рабочая среда, где ты будешь писать код, запускать ячейки, тестировать решения и фиксировать прогресс.

Альтернативой Jupyter Notebooks служит Google Colab.

> **Google Colab** — это облачная среда, аналогичная Jupyter Notebook, где можно:

- Работать над проектами без установки Python на свой компьютер,
- Использовать бесплатный GPU и TPU для ускорения вычислений,
- Легко делиться проектами и совместно работать с другими.

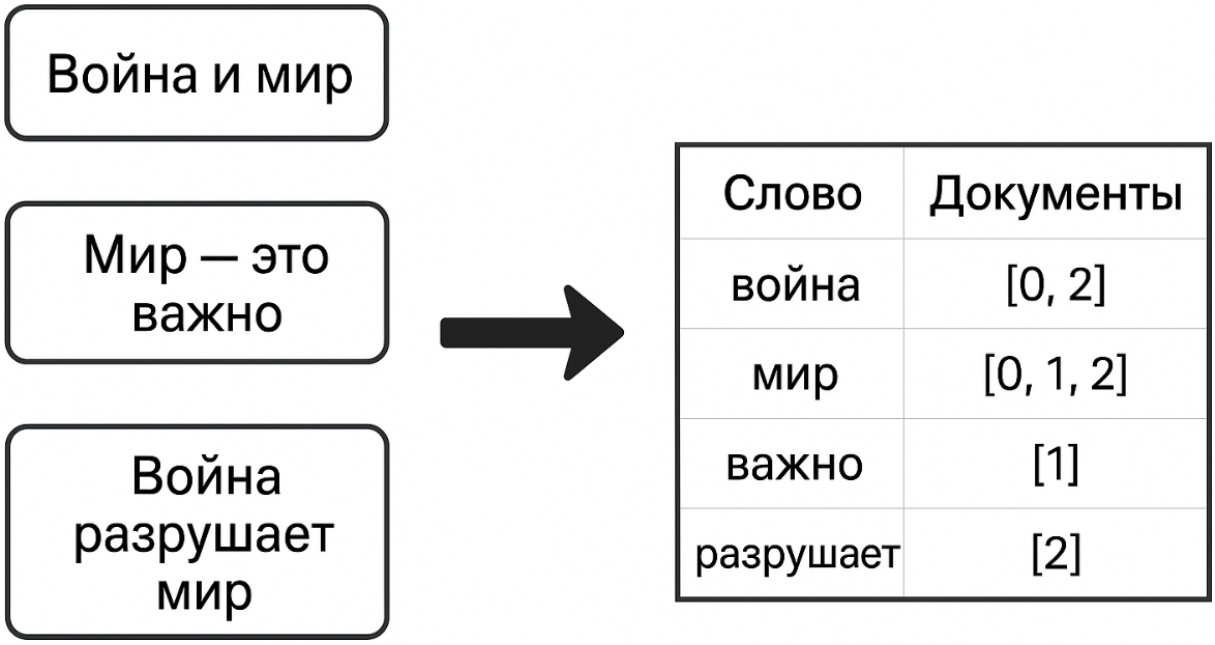
Сегодняшние задания можно выполнять в Jupyter Lab или Google Colab в зависимости от твоих предпочтений и доступности ресурсов.

1. Инвертированный индекс

> **Информационный поиск** (Information Retrieval) — это область компьютерных наук, которая изучает методы поиска нужной информации в больших коллекциях данных. Классический пример — поисковые системы в интернете, которые позволяют найти документы, статьи или веб-страницы по ключевым словам.

Цель информационного поиска — не просто найти документы, содержащие слово, но и оценить их релевантность запросу пользователя. Одним из основных инструментов в информационном поиске является инвертированный индекс.

> **Инвертированный индекс** — это структура данных, которая для каждого уникального слова хранит список документов или частей текста, где это слово встречается. Проще говоря, он «переворачивает» данные: вместо списка документов с их словами создается список слов с привязкой к документам.



Задание 1.1. Разделение на предложения

- Считай текст книги «Война и мир» из .txt-файла.
- Раздели текст на предложения по знакам . ! ?, удали пробелы и пустые строки.
- Посмотри на полученные предложения и выведи не менее двух примеров, когда разбиение некорректное.
- Раздели текст на предложения с помощью библиотеки razdel.
- Выведи количество предложений, полученных с помощью razdel.

Задание 1.2. Очистка текста

- Очисти текст от знаков препинания и приведи все слова к нижнему регистру.
- Разбей каждое предложение на список слов.
- Удали стоп-слова из файла.
- Удали все пустые предложения (те, что после очистки не содержат слов).

- Выведи:
  - Количество слов в самом длинном предложении,
  - Общее количество предложений, оставшихся после очистки.

Задание 1.3. Построение инвертированного индекса

Реализуй **инвертированный индекс** для поиска слов в предложениях с помощью класса *InvertedIndex*. Для этого:

- Реализуй метод *build_index*, который **строит индекс из списка документов**, где каждый документ — это список слов.
  - Для каждого уникального слова сохраняй список id документов (предложений), в которых оно встречается.
  - Используй атрибут *word2doc* для хранения данных.
- Реализуй метод *save_index*, чтобы **сохранять индекс в файл** (например, с помощью *pickle*) для последующего использования.
- Реализуй метод *load_index*, чтобы **загружать индекс из файла**.
- Реализуй метод *__ len__*, который **возвращает количество уникальных слов в индексе**.

Протестируй работу индекса:

- Построй индекс на списке предложений книги.
- Сохрани индекс в файл.
- Загрузи индекс из файла в новый объект.
- Выведи количество уникальных слов в загруженном индексе.

Задание 1.4. Поиск по индексу

- Реализуй метод *query* класса *InvertedIndex*, который выполняет поиск слов в индексе.

Метод должен принимать **список слов** и возвращать список id предложений, в которых встречаются **все указанные слова** (операция «и»).

Результат должен быть **отсортирован по возрастанию** id предложения.

- Проверь работу метода query на следующих примерах:
  - Слово: университет;
  - Слова: война и мир;
  - Слово: школа.

Бонусное задание. TF-IDF

Построив **инвертированный индекс**, мы научились быстро находить документы, содержащие определенные слова. Однако этого недостаточно для реального поиска. Представь, что ты ищешь слово «мир» в романе «Война и мир». Индекс вернет сотни предложений, но не все они будут одинаково полезны.

В поисковых системах важно не только найти документы, где встречается слово, но и **оценить, насколько это слово важно именно в данном документе**. Для этого используется одна из ключевых идей информационного поиска — **TF-IDF** (Term Frequency — Inverse Document Frequency).

- **TF** показывает, насколько часто слово встречается в документе.
- **IDF** снижает вес слов, которые встречаются слишком часто в коллекции (например, «и», «он», "the").
- В итоге TF-IDF позволяет отличать «шумовые» слова от действительно значимых.

Выполни задания ниже, чтобы понять, как работает TF-IDF. Задания не являются обязательными и не влияют на успешность выполнения проекта, но точно дадут прирост к твоим практическим навыкам.

Задание 2.1. Написание формулы TF-IDF

- Прочитай, как работает метод TF-IDF.
- Разберись, что означают:
  - TF — частота слова в документе,
  - IDF — обратная частота документа,
  - TF-IDF — произведение этих величин.
- Запиши формулу TF-IDF с помощью Markdown-разметки.

Задание 2.2. Реализация TF

- Создай класс *TfidfIndex*, унаследованный от *InvertedIndex*.
- Реализуй в нем метод *_tf*.
- Проверь работу метода на первом предложении книги.

Задание 2.3. Реализация IDF

В классе *TfidfIndex* реализуй метод *_idf*. Выведи топ-5 слов с самым маленьким значением.

Задание 2.4. Реализация TF-IDF

В классе *TfidfIndex* реализуй метод *tfidf*, который принимает список документов и возвращает матрицу TF-IDF. Рассчитай TF-IDF для всей книги. Выведи полученные значения для слов *война* и *мир* для первого предложения.

Глава 4

Сдача работы и проверка

Сохрани решение в файле *01-assignment.ipynb*.
Загрузи изменения на Git в ветку develop.